Practical 1:

Part A

1. Vending Machine

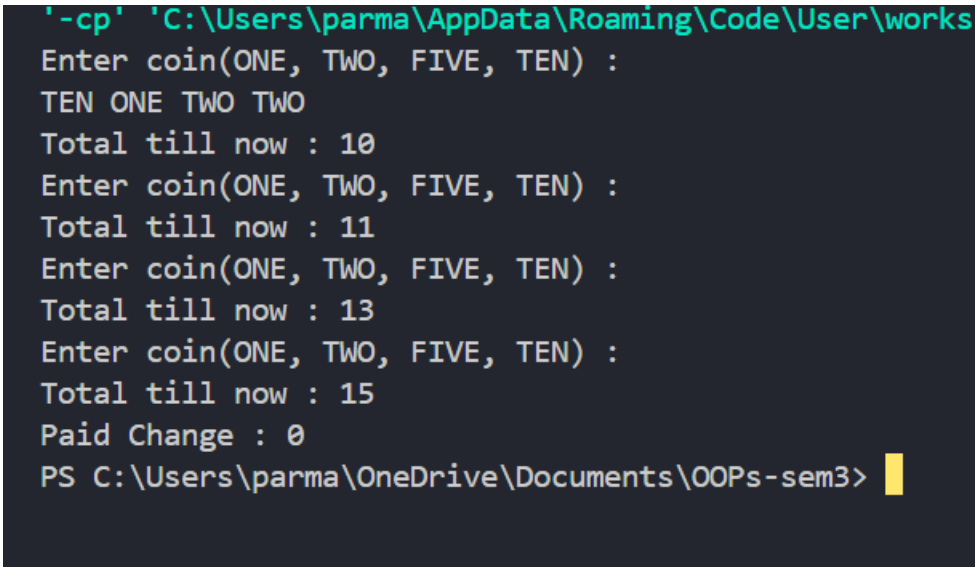
Problem Statement:

(a) Define an enum Coin with constants ONE, TWO, FIVE, TEN.
(b) In main, set a snack price of 15 and a running total of 0; create a Scanner.
(c) Loop: read a coin name, use a switch expression to convert the Coin to its value
(ONE→1, TWO→2, FIVE→5, TEN→10), add it to the total, and print the total so far.
(d) Stop the loop once the total reaches 15 or more.
(e) Print the change to return (total − 15).
Expected output: TEN then FIVE prints “Paid. Change: 0”; TEN then TEN prints “Paid. Change: 5”.

Solution:

```java
import java.util.Scanner;
public class vendingMachine {
    enum Coin {
        ONE, TWO, FIVE, TEN;
    }
    Run | Debug
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        int price = 15;
        int total = 0;
        while(total<price)
        {
            System.out.println(x: "Enter coin(ONE, TWO, FIVE, TEN) : ");
            Coin coin = Coin.valueOf(sc.next().toUpperCase());
            int value = switch(coin)
            {
                case ONE -> 1;
                case TWO -> 2;
                case FIVE -> 5;
                case TEN -> 10;
            };
            total+=value;
            System.out.println("Total till now : " + total);
        }
        System.out.println("Paid Change : " + (total-price));
        sc.close();
    }
}
```

Output:



2. Toll Booth

Problem Statement:

(a) Define a record Vehicle(String number, String type).
(b) In main, keep a running toll total and three counters (bike, car, truck).
(c) Loop until the user types “done” for the number: build a Vehicle, use a switch expression on its type for the toll (bike→20, car→50, truck→150), add to the total, and increment that type’s counter.
(d) After the loop, print the total toll and the type with the highest count.
Expected output: two cars + one bike → “Total toll: 120”, “Most frequent: car”.

Solution:

```java
import java.util.Scanner;
public class TollBooth {
    record Vehicle(String number, String type){}
    Run | Debug
    public static void main(String[] args)
    {
        Scanner s = new Scanner(System.in);
        int total=0;
        int bikec=0, carc=0, truckc=0;
        while(true)
        {
            System.out.println(x: "Enter Vehicle Number (or done for exit) : ");
            String number = s.next();
            if(number.equalsIgnoreCase(anotherString: "done"))
                break;
            System.out.print(s: "Enter Vehicle Type : ");
            String type = s.next().toLowerCase();
            Vehicle v = new Vehicle(number, type);
            int toll= switch(v.type())
            {
                case "bike" -> 20;
                case "car" -> 50;
                case "truck" -> 150;
                default -> 0;
            };
            total+=toll;
            switch(v.type())
            {
                case "bike" -> bikec++;
                case "car" -> carc++;
                case "truck" -> truckc++;
            }
        }
        System.out.println("Total toll : "+total);
       if(bikec>=carc && bikec>=truckc)
        System.out.println(x: "Most Frequent : Bike");
    else if(carc>=bikec && carc>=truckc)
        System.out.println(x: "Most Frequent : Car");
    else System.out.println(x: "Most Frequent : Truck");
    s.close();
    }
}
```

Output:

```
Enter Vehicle Number (or done for exit) :
GJO7BH064
Enter Vehicle Type : CAR
Enter Vehicle Number (or done for exit) :
MH098HJ
Enter Vehicle Type : TRUCK
Enter Vehicle Number (or done for exit) :
DONE
Total toll : 200
Most Frequent : Car
```

3. Rock-Paper-Scissors-Lizard-Spock

Problem Statement:

(a) Define an enum Move with ROCK, PAPER, SCISSORS, LIZARD, SPOCK.

(b) Write winner(Move a, Move b) returning 1 if a beats b, -1 if b beats a, 0 for a tie —
use a switch expression listing what each move beats (Scissors cut Paper, Paper covers Rock, Rock crushes Lizard, Lizard poisons Spock, Spock smashes Scissors,
and the five mirror cases).
(c) In main, play 5 rounds: pick a random computer Move, read the player's Move, call
winner(), print the round result and keep score.
(d) After 5 rounds, print the overall winner.
Expected output: each round prints both moves and the round winner; ends with a
"You win 3–2" summary.

Solution:

```java
import java.util.*;
public class RPSLS {
    enum Move {
        ROCK, PAPER, SCISSORS, LIZARD, SPOCK
    }
    static int winner(Move a, Move b) {
        if (a == b) return 0;

        return switch (a) {
            case ROCK -> (b == Move.SCISSORS || b == Move.LIZARD) ? 1 : -1;
            case PAPER -> (b == Move.ROCK || b == Move.SPOCK) ? 1 : -1;
            case SCISSORS -> (b == Move.PAPER || b == Move.LIZARD) ? 1 : -1;
            case LIZARD -> (b == Move.SPOCK || b == Move.PAPER) ? 1 : -1;
            case SPOCK -> (b == Move.SCISSORS || b == Move.ROCK) ? 1 : -1;
        };
    }
    Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Random rand = new Random();
        int playerScore = 0;
        int computerScore = 0;
        System.out.println(x: "Moves: ROCK, PAPER, SCISSORS, LIZARD, SPOCK");
        for (int i = 1; i <= 5; i++) {
            System.out.print("Round " + i + " - Enter your move: ");
            Move player = Move.valueOf(sc.next().toUpperCase());
            Move computer = Move.values()[rand.nextInt(bound: 5)];
            System.out.println("Player: " + player);
            System.out.println("Computer: " + computer);
            int result = winner(player, computer);
            if (result == 1) {
                System.out.println(x: "You win this round!");
                playerScore++;
            } else if (result == -1) {
                System.out.println(x: "Computer wins this round!");
                computerScore++;
            } else {
                System.out.println(x: "Round is a tie!");
            }
            System.out.println();
        }
        System.out.println(x: "Final Score");
        System.out.println("You: " + playerScore);
        System.out.println("Computer: " + computerScore);
        if (playerScore > computerScore)
            System.out.println("You win " + playerScore + "-" + computerScore);
        else if (computerScore > playerScore)
            System.out.println("Computer wins " + computerScore + "-" + playerScore);
        else
            System.out.println("Match Draw " + playerScore + "-" + computerScore);
        sc.close();
    }
}
```

Output:

```
Moves: ROCK, PAPER, SCISSORS, LIZARD, SPOCK
Round 1 - Enter your move: ROCK
PAPER
SCISSORS
LIZARD
Player: ROCK
Computer: LIZARD
You win this round!

Round 2 - Enter your move: Player: PAPER
Computer: LIZARD
Computer wins this round!

Round 3 - Enter your move: Player: SCISSORS
Computer: SPOCK
Computer wins this round!

Round 4 - Enter your move: Player: LIZARD
Computer: SPOCK
You win this round!

Round 5 - Enter your move: SPOCK
Player: SPOCK
Computer: LIZARD
Computer wins this round!

Final Score
You: 2
Computer: 3
Computer wins 3-2
```

Part B: — Project Work (Hour 2): MiniBank

Problem Statement:

MiniBank is the console banking application you will build across the whole semester. In
this first lab you set up your tools and create the project and its menu shell — the screen the
user navigates. Every later lab attaches one real banking feature behind these menu options.

1. Set up your environment: open the project in GitHub Codespaces (or install JDK 17+ and

an IDE locally), and create a private GitHub repository named oop-minibank-<rollno>;
add the faculty as a collaborator.
2. Create a public class named MiniBank that contains the main method.
3. Define a record named BankInfo with two String fields, name and branch (a record is a
short, read-only class; Java auto-generates its constructor and field accessors). Print one
BankInfo object as the application header.
4. Define an enum named MenuOption with the constants OPEN_ACCOUNT, DEPOSIT,
WITHDRAW, TRANSFER and EXIT — these are the actions MiniBank will offer.
5. Display the numbered menu to the user and read the chosen number with a Scanner.
6. Use a switch expression on the choice to print a short placeholder for that option (for
example, “Deposit — to be implemented in a later lab”). Keep showing the menu in a
loop until the user selects EXIT, then print a goodbye message.

Solution:

```
import java.util.Scanner;

public class MiniBank {

  record BankInfo(String name, String branch) {}

  enum MenuOption {
    OPEN_ACCOUNT,
    DEPOSIT,
    WITHDRAW,
    TRANSFER,
    EXIT
  }

  public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);
    BankInfo bank = new BankInfo("MiniBank", "Main Branch");
```

```
        System.out.println("================================");
        System.out.println(bank.name() + " - " + bank.branch());
        System.out.println("================================");

        while (true) {
          System.out.println("\nMenu");
          System.out.println("1. Open Account");
          System.out.println("2. Deposit");
          System.out.println("3. Withdraw");
          System.out.println("4. Transfer");
          System.out.println("5. Exit");
          System.out.print("Enter your choice: ");

          int choice = sc.nextInt();

          MenuOption option = switch (choice) {
            case 1 -> MenuOption.OPEN_ACCOUNT;
            case 2 -> MenuOption.DEPOSIT;
            case 3 -> MenuOption.WITHDRAW;
            case 4 -> MenuOption.TRANSFER;
            case 5 -> MenuOption.EXIT;
            default -> null;
          };

          if (option == null) {
            System.out.println("Invalid choice!");
            continue;
          }

          switch (option) {
            case OPEN_ACCOUNT ->
                System.out.println("Open Account - To be implemented in
    a later lab.");
            case DEPOSIT ->
```

```
                System.out.println("Deposit - To be implemented in a later
lab.");
            case WITHDRAW ->
                System.out.println("Withdraw - To be implemented in a
later lab.");
            case TRANSFER ->
                System.out.println("Transfer - To be implemented in a later
lab.");
            case EXIT -> {
              System.out.println("Thank you for using MiniBank!");
              sc.close();
              return;
            }
          }
        }
      }
}
```

Output:

```
================================
MiniBank - Main Branch
================================

Menu
1. Open Account
2. Deposit
3. Withdraw
4. Transfer
5. Exit
Enter your choice: 1
Open Account - To be implemented in a later lab.

Menu
1. Open Account
2. Deposit
3. Withdraw
4. Transfer
5. Exit
Enter your choice: 5
Thank you for using MiniBank!
```